

Generative Models, QAOA, and Quantum Physics-Informed NNs

EVA: Quantum Machine Learning – Lecture 12

Pavel Sulimov

Spring 2026

ZHAW Zürcher Hochschule für Angewandte Wissenschaften

1. Combinatorial optimization and Hamiltonians
2. QAOA: Quantum Approximate Optimization Algorithm
3. Quantum Born machines and generative modeling
4. Quantum GANs
5. Quantum Physics-Informed Neural Networks (QPINNs)
6. Quantum utility: IBM 127-qubit experiments

Part 1. Combinatorial optimization and Hamiltonians

From combinatorial problems to Ising Hamiltonians

Many combinatorial optimization problems can be mapped to finding the ground state of a classical Ising Hamiltonian:

$$H_C = \sum_{(i,j) \in E} J_{ij} Z_i Z_j + \sum_i h_i Z_i,$$

where $Z_i \in \{+1, -1\}$ are classical spin variables, J_{ij} are couplings, and h_i are local fields.

In the quantum context, Z_i becomes the Pauli-Z operator acting on qubit i . Minimizing the energy $\langle H_C \rangle$ over quantum states gives the quantum ground state energy (which lower bounds the classical minimum).

QUBO (Quadratic Unconstrained Binary Optimization): Problems formulated over binary variables $x_i \in \{0, 1\}$ map to Ising form via $x_i = (1 - Z_i)/2$:

$$\min_{x \in \{0,1\}^n} x^\top Q x \iff \min_z z^\top \tilde{Q} z + c.$$

MaxCut: a canonical example

MaxCut problem: given a graph $G = (V, E)$, partition the vertices into two sets S and $\bar{S}$ to maximize the number of edges cut (one endpoint in each set).

MaxCut maps directly to the Ising Hamiltonian:

$$H_C = - \sum_{(i,j) \in E} \frac{1 - Z_i Z_j}{2} = -\frac{|E|}{2} + \frac{1}{2} \sum_{(i,j) \in E} Z_i Z_j.$$

For the triangle graph (3 nodes, all 3 edges):

$$H_C = -\frac{1}{2}(3 - Z_0 Z_1 - Z_1 Z_2 - Z_0 Z_2).$$

Maximum cut value is 2 (achieved by any partition of the triangle). The optimal bitstrings are $|010\rangle, |101\rangle, |001\rangle, |110\rangle, |100\rangle, |011\rangle$.

lecture_12_demo.ipynb, Demo 1: MaxCut Hamiltonian construction.

Part 2. QAOA

QAOA: the variational approach to optimization

The Quantum Approximate Optimization Algorithm (Farhi, Goldstone, Gutmann 2014) uses a variational ansatz inspired by adiabatic quantum computation.

The QAOA ansatz with p layers:

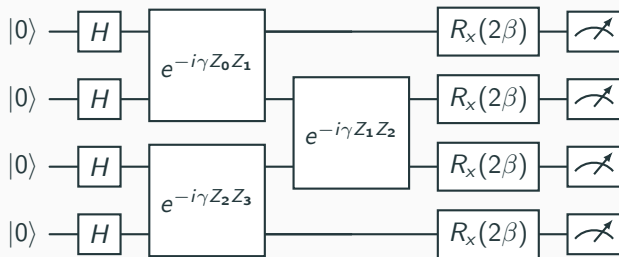
$$|\psi(\boldsymbol{\gamma}, \boldsymbol{\beta})\rangle = \prod_{k=1}^p e^{-i\beta_k H_M} e^{-i\gamma_k H_C} |+\rangle^{\otimes n},$$

where:

- H_C is the problem (cost) Hamiltonian
- $H_M = -\sum_i X_i$ is the mixer Hamiltonian (drives transitions between solutions)
- $|+\rangle^{\otimes n}$ is the uniform superposition (equal weight on all solutions)
- $\boldsymbol{\gamma} = (\gamma_1, \dots, \gamma_p)$ and $\boldsymbol{\beta} = (\beta_1, \dots, \beta_p)$ are optimized classically

QAOA: circuit structure

One QAOA layer ($p = 1$) for MaxCut on 4 qubits (ring graph):



Each $e^{-i\gamma Z_i Z_j}$ is a two-qubit phase gate (implemented as CNOT + $R_z(2\gamma)$ + CNOT). The mixer $e^{-i\beta H_M} = \bigotimes_i R_x(2\beta)$.

lecture_12_demo.ipynb, Demo 2: QAOA $p = 1$ and $p = 2$ for MaxCut.

QAOA: approximation ratio and depth

The QAOA approximation ratio $r_p = \langle H_C \rangle / H_C^*$ measures how close the expected value is to the true optimum H_C^* .

Key results:

- $p = 1$ QAOA guarantees $r_1 \geq 0.6924$ for MaxCut on 3-regular graphs (Farhi et al. 2014)
- $r_p \rightarrow 1$ as $p \rightarrow \infty$ (QAOA converges to the exact solution)
- the convergence rate is problem-dependent; hard instances require large p
- for $p \geq n$, QAOA can exactly solve MaxCut (but the circuit is deep)

On NISQ hardware, $p \leq 3-5$ is typically feasible. The practical question is whether low- p QAOA outperforms classical heuristics (greedy, simulated annealing) on the same hardware budget. Current evidence is mixed: classical solvers remain competitive for most instances accessible to near-term hardware.

Connection to adiabatic quantum computation

QAOA can be understood as a Trotterized adiabatic evolution.

Adiabatic quantum computation (AQC) slowly evolves the Hamiltonian:

$$H(t) = \left(1 - \frac{t}{T}\right) H_M + \frac{t}{T} H_C, \quad t \in [0, T].$$

Starting in the ground state of H_M (the $|+\rangle^{\otimes n}$ state) and evolving slowly to $t = T$ gives the ground state of H_C .

QAOA approximates this evolution by dividing it into p discrete steps: each layer applies $e^{-i\gamma_k H_C}$ (cost evolution) then $e^{-i\beta_k H_M}$ (mixer evolution). For large p and optimal γ_k, β_k , this converges to the adiabatic path.

In practice, the optimal angles for small p differ from the adiabatic schedule and must be found by classical optimization (COBYLA, Nelder-Mead, SPSA).

Part 3. Quantum generative models

Quantum Born machines

A Quantum Circuit Born Machine (QCBM) uses a parameterized quantum circuit $U(\theta)$ to generate samples from a target distribution $p^*(x)$.

The generated distribution is:

$$p_\theta(x) = |\langle x | U(\theta) | 0^n \rangle|^2,$$

the probability of observing bitstring x when measuring the circuit output in the computational basis. By the Born rule, this is a valid probability distribution (non-negative, sums to 1).

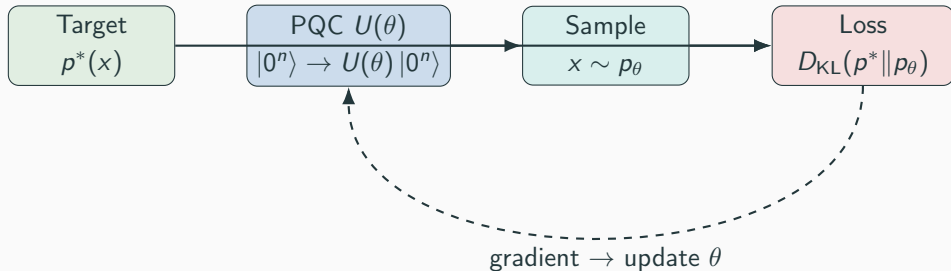
Training objective: minimize a divergence between p_θ and p^* :

$$\mathcal{L}(\theta) = D_{\text{KL}}(p^* \| p_\theta) = \sum_x p^*(x) \log \frac{p^*(x)}{p_\theta(x)}.$$

Gradients of $p_\theta(x)$ with respect to θ are computed via the parameter-shift rule (same as for expectation values).

lecture_12_demo.ipynb, Demo 3: QCBM learning a discretized Gaussian.

QCBM: architecture and training



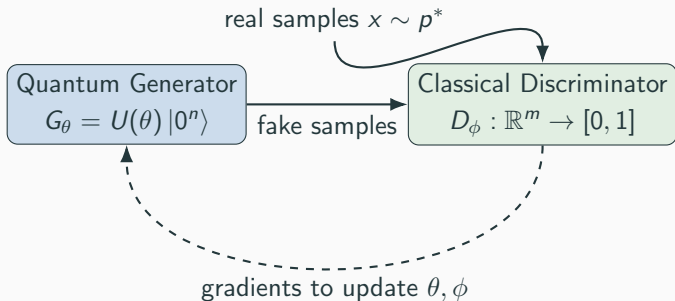
For discrete target distributions, $p^*(x)$ is specified by a histogram. For continuous distributions, p^* is discretized over 2^n bins.

Limitation: the QCBM can only represent distributions over bit strings. Continuous distributions require discretization or additional post-processing. On current hardware, the output distribution is noisy and requires shot-averaging.

Part 4. Quantum GANs

Quantum GANs: structure

Dallaire-Demers and Killoran (2018) introduced the Quantum GAN (qGAN): a generative adversarial network where the generator is a parameterized quantum circuit.



The generator samples from $p_\theta(x) = |\langle x | U(\theta) | 0^n \rangle|^2$. The discriminator is a classical neural network.

qGAN training and stability

Training quantum GANs faces the same challenges as classical GANs plus quantum-specific ones:

Challenge	Mitigation
Mode collapse (generator ignores most modes)	Wasserstein loss or MMD loss instead of JS
Discriminator overpowers generator	Balance update frequencies
Shot noise in gradient estimates	Use large shot counts or backprop mode
Barren plateaus in generator	Use shallow circuits, local cost
Non-differentiable discrete outputs	Use expectation values, not bitstring samples

In practice, qGANs have been demonstrated on 1D and 2D synthetic distributions. Scaling to high-dimensional real-world data (images, text) remains an open research problem.

`lecture_12_demo.ipynb`, Demo 4: qGAN on a 1D mixture distribution.

Part 5. Quantum Physics-Informed Neural Networks

Classical PINNs: recap

Physics-Informed Neural Networks (Raissi, Perdikaris, Karniadakis 2019) encode physical laws directly into the training loss.

For a PDE $\mathcal{N}[u](x, t) = 0$ (where $\mathcal{N}$ is a differential operator), the PINN loss is:

$$\mathcal{L} = \underbrace{\frac{1}{N_b} \sum_{(x,t) \in \partial\Omega} \|u_\theta - u_{\text{BC}}\|^2}_{\text{boundary conditions}} + \lambda \underbrace{\frac{1}{N_r} \sum_{(x,t) \in \Omega} \|\mathcal{N}[u_\theta]\|^2}_{\text{PDE residual}}.$$

The PDE residual enforces the governing equation at collocation points without requiring a labeled solution. Gradients $\partial u / \partial x$ are computed via automatic differentiation.

QPINNs: replacing the NN with a PQC

In a Quantum PINN (QPINN), the function approximator $u_\theta(x, t)$ is a parameterized quantum circuit:

$$u_\theta(x, t) = \langle 0^n | U^\dagger(\theta, x, t) O U(\theta, x, t) | 0^n \rangle.$$

The input (x, t) is injected via angle encoding. The output is an expectation value of an observable O . Gradients $\partial u_\theta / \partial x$ are computed via the parameter-shift rule (treating x as a circuit parameter):

$$\frac{\partial u_\theta}{\partial x_k} = \frac{1}{2} \left[u_\theta \left(x_k + \frac{\pi}{2} \right) - u_\theta \left(x_k - \frac{\pi}{2} \right) \right].$$

The PINN loss is then evaluated and minimized over θ using classical optimizers.

QPINN: worked example

Consider the simplest ODE: $\frac{du}{dx} = -u$, with solution $u(x) = e^{-x}$.

QPINN setup:

- circuit: $U(\theta, x) = R_y(\theta_1)R_z(x)R_y(\theta_2)$ on 1 qubit
- output: $u_\theta(x) = \langle Z \rangle$
- loss: $\mathcal{L} = \frac{1}{N_r} \sum_i \left(\left. \frac{\partial u_\theta}{\partial x} \right|_{x_i} + u_\theta(x_i) \right)^2 + (u_\theta(0) - 1)^2$

The PDE residual is $\partial u / \partial x + u = 0$. The initial condition $u(0) = 1$ is the boundary term.

With only 2 trainable parameters (θ_1, θ_2) , the QPINN approximates the exponential decay over $x \in [0, 3]$. The approximation is limited by the expressibility of the 1-qubit circuit.

lecture_12_demo.ipynb, Demo 3: QPINN for $u' = -u$.

QPINN: challenges and practical status

Challenge	Detail
Barren plateaus	The PINN loss is a global cost (sum over all collocation points). Global costs exhibit exponentially small gradients for deep circuits.
Circuit depth for complex solutions	Approximating functions with sharp features (e.g. shocks) requires high-frequency harmonics, hence many re-uploading layers.
Gradient computation cost	Each PDE residual term requires evaluating $\partial u / \partial x_k$ via parameter shift: 2 circuit calls per spatial dimension per collocation point.
Hardware noise	Accumulation of gate errors over many circuit calls in the optimization loop biases the gradient estimates.

Recent hybrid approaches (classical NN + quantum circuit) reduce the circuit depth requirement while retaining some quantum structure.

Part 6. Quantum utility

IBM quantum utility experiments

Kim et al. (Nature, 2023) demonstrated **quantum utility**: a computation that is difficult to brute-force classically but tractable on a 127-qubit IBM Eagle processor.

Task: simulate the time evolution of a 127-qubit Ising model $H = J \sum_{\langle i,j \rangle} Z_i Z_j + h \sum_i X_i$ to measure magnetization $\langle Z \rangle$ at late times.

Results:

- direct statevector simulation is impossible (2^{127} amplitudes)
- matrix product state (MPS) simulation with bond dimension χ fails to converge for the target circuit depth on this specific geometry
- the quantum hardware (with error mitigation via Pauli noise learning) produced results consistent with approximate classical methods

Terminology: “utility” means the hardware is a useful computational tool even without error correction. It does not claim “advantage” (better than all classical methods) or “supremacy” (no classical algorithm can solve it).

Utility $\neq$ advantage $\neq$ supremacy

Three distinct claims in quantum computing:

Term	Meaning	Established?
Quantum supremacy	No classical computer can match	Disputed (RCS, boson sampling)
Quantum advantage	Better performance on a useful task	Not yet for practical ML tasks
Quantum utility	Useful in practice even without advantage	Yes (Kim et al. 2023)

For QML specifically:

- rigorous advantage proofs exist for contrived, artificial tasks
- practical advantage on real ML benchmarks has not been demonstrated
- utility-scale computations (error-mitigated, 100+ qubits) are available now via IBM cloud and can be relevant for quantum-native data (chemistry, physics)

Summary

Topic	Key takeaway
Ising/QUBO	Combinatorial problems map to ground-state search
QAOA	Variational Trotterization of adiabatic evolution; p layers, $2p$ params
Approximation ratio	$r_1 \geq 0.6924$ for MaxCut/3-regular; classical still competitive
Born machine	PQC output probabilities as generative distribution; KL loss
qGAN	Quantum generator + classical discriminator; same stability issues as GAN
QPINN	PQC replaces NN in PINN; gradients via parameter shift on x
QPINN challenges	Global cost + barren plateaus + 2 circuit calls per gradient
Quantum utility	IBM 127-qubit Ising simulation; useful tool without full advantage

Lecture 13 addresses **noise, mixed states, and error mitigation** – the tools that make utility-scale experiments possible.